

TRAINING & REFERENCE

murach's Java servlets and JSP (Chapter 2)

Andrea Steelman
Joel Murach



MIKE MURACH & ASSOCIATES, INC.

1-800-221-5528 • (559) 440-9071 • Fax: (559) 440-0963

murachbooks@murach.com • www.murach.com

Copyright © 2005 Mike Murach & Associates. All rights reserved.

Book contents

Introduction	xi
Section 1 Introduction to servlet and JSP programming	
Chapter 1 An introduction to web programming	3
Chapter 2 How to install and use Tomcat	29
Chapter 3 A crash course in HTML	61
Section 2 The essence of servlet and JSP programming	
Chapter 4 How to develop JavaServer Pages	101
Chapter 5 How to develop servlets	141
Chapter 6 How to structure a web application	167
Chapter 7 How to work with sessions and cookies	203
Chapter 8 How to create and use JavaBeans	243
Chapter 9 How to work with custom JSP tags	267
Section 3 The essential database skills	
Chapter 10 How to use MySQL to work with a database	307
Chapter 11 How to use Java to work with a database	333
Section 4 Advanced servlet and JSP skills	
Chapter 12 How to use JavaMail to send email	375
Chapter 13 How to use SSL to work with a secure connection	397
Chapter 14 How to restrict access to a web resource	417
Chapter 15 How to work with HTTP requests and responses	437
Chapter 16 How to work with XML	465
Chapter 17 An introduction to Enterprise JavaBeans	503
Section 5 The Music Store web site	
Chapter 18 An introduction to the Music Store web site	523
Chapter 19 The Download application	545
Chapter 20 The Shopping Cart application	557
Chapter 21 The Admin application	579
Appendixes	
Appendix A How to install the software and applications for this book	599
Index	614

How to install and use Tomcat

As you learned in the last chapter, Tomcat can be used as both the web server and the servlet and JSP engine for web applications. Now, in this chapter, you'll learn how to install and use Tomcat on your own PC. Along the way, you should take the time to install Tomcat.

Before you start this chapter, we recommend that you install the other software that's on the book's CD as summarized in appendix A. If you've used our Beginning Java 2 book to get started with Java, most of this software should already be installed. Once you have all of it installed, including Tomcat, you'll be ready to start developing servlets and JSPs on your own PC.

If you're going to work in a client/server environment, of course, the web server and the servlet and JSP engine may already be installed on the server. In that case, you won't need to install it on your client computer. If, on the other hand, you're the one who is supposed to install Tomcat on the server, the procedure is similar to the one in this chapter, and you can get the extra information that you need from one of the text files that's described in this chapter.

How to install and configure Tomcat	30
How to install Tomcat on your own PC	30
A summary of Tomcat's directories and files	32
How to set the JAVA_HOME environment variable	34
How to change the memory settings for Tomcat's batch files	36
How to turn on servlet reloading	38
How to use Tomcat	40
How to start and stop Tomcat	40
How to view a web page	42
How to view a directory listing	44
A quick guide to troubleshooting	46
Two common Tomcat problems and how to solve them	46
How to change the port that's used by Tomcat	48
How to deploy and run a web application	50
How to deploy a web application	50
How to view and run a web application	50
A summary of the directories and files for a web application	52
An introduction to the web.xml file for a web application	54
A summary of the directories and files for the book applications	56
Perspective	58

How to install and configure Tomcat

Although this topic shows how to install and configure Tomcat on a Windows system, similar concepts apply to all operating systems. As a result, if you're using another operating system, you may be able to get the installation done by following the procedures in this topic. Otherwise, you can go to the Tomcat web site to get information about installing Tomcat on your system.

How to install Tomcat on your own PC

Figure 2-1 shows how to install version 4.0 of Tomcat. If you're using Windows, the easiest way to install it is to use the CD that comes with this book. After you navigate to the Tomcat 4.0 directory on the CD, you use a zip program like WinZip to extract the files from the zip file named Jakarta-Tomcat-4.0.1.

The other alternative is to download the current version of Tomcat from the Tomcat web site. However, since the Apache Software Foundation is continually updating this web site, the procedure in this figure may be out of date by the time you read this. As a result, you may have to do some searching to find the current version of Tomcat.

After you've installed Tomcat, its top-level directory is jakarta-tomcat-4.0.X. This can be referred to as the *Tomcat home directory*. To make it easier to refer to this directory later on, we recommend that you change its name to tomcat. This is the second last step in both of the procedures in this figure.

The last step in both procedures is to copy (not move) the servlet.jar file that's in the tomcat\common\lib directory to the jre\lib\ext directory for the Java SDK. That way, the servlet classes that come with Tomcat are available to the SDK. Although you could get the same result by adding the tomcat\common\lib directory to the classpath of your system, copying the files is a better way to do this.

The Apache web site address

`www.apache.org`

How to install Tomcat from the CD that comes with the book

1. Put the CD that comes with this book into your CD drive, and navigate to the Tomcat 4.0 directory.
2. Use a zip program such as WinZip to extract the files from the zip file named jakarta-tomcat-4.0.1. If you don't have WinZip, you can get it free from www.winzip.com.
3. If necessary, move the jakarta-tomcat directory to the C drive.
4. Rename the jakarta-tomcat directory to tomcat.
5. Copy the servlet.jar file from the C:\tomcat\common\lib directory to the C:\j2sdk1.4.0\jre\lib\ext directory.

How to download and install Tomcat from the Apache web site

1. Go to the Apache web site (www.apache.org).
2. Navigate to the Apache Projects heading and click on the Jakarta link.
3. Navigate to the SubProjects heading and click on the Tomcat link.
4. Navigate to the Download heading and select the Binaries link.
5. Navigate to the Release Builds heading and select the most recent version of Tomcat.
6. Select the bin directory link and download the jakarta-tomcat-4.0.X zip file.
7. Save the zip file to your hard disk.
8. Use a zip program such as WinZip to extract the files from the zip file. If you don't have WinZip, you can get it free from www.winzip.com.
9. If necessary, move the jakarta-tomcat directory to your C drive.
10. Rename the jakarta-tomcat directory to tomcat.
11. Copy the servlet.jar file from the C:\tomcat\common\lib directory to the C:\j2sdk1.4.0\jre\lib\ext directory.

Description

- The directory that holds the files for Tomcat is known as the *Tomcat home directory*. By default, this directory is jakarta-tomcat-4.0.X. However, to save yourself typing, we recommend that you rename this directory to tomcat.
- The servlet.jar file contains the Java classes that you need for developing servlets. By copying this JAR file from the tomcat subdirectory to the SDK subdirectory, you make the classes available to the SDK.

A summary of Tomcat's directories and files

Figure 2-2 shows the directories and files for the Tomcat home directory, which has been changed to `c:\tomcat`. By default, this directory has nine subdirectories. Of these, the most important are the `bin`, `conf`, `log`, `webapps`, and `work` directories.

The `bin` directory holds the binary files that let you start and stop Tomcat. You'll learn how to do that later in this chapter.

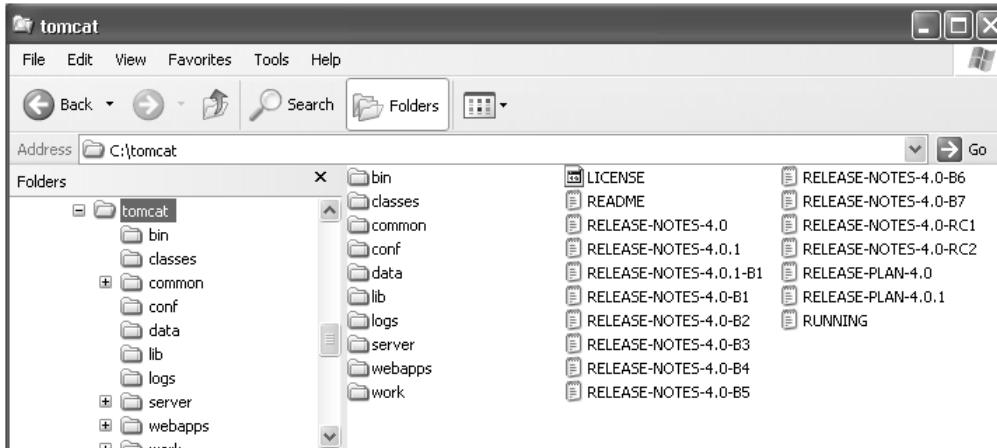
The `webapps` directory contains some servlet and JSP applications that come with Tomcat. You can run these applications to make sure Tomcat has been installed properly. Later, when you develop your own web applications, you will store them under this directory.

The `conf` directory contains some XML files that you may need to edit as you configure Tomcat, while the `log` directory contains text files that Tomcat uses to log its operations. You can open these XML and log files in a text editor to learn more about how Tomcat works.

The `work` directory is used by Tomcat to store the servlet files that the JSP engine creates. In chapter 4, you'll learn how to view these files in a text editor to help debug your code.

In the Tomcat home directory, you can also find two text files. The `readme` text file contains some general information about Tomcat. The `running` text file contains more information about installing, starting, and stopping Tomcat. Although most of this information is presented in this chapter, you can check the `running` file if you encounter any problems installing and running Tomcat. In particular, this file includes directions for Unix users, and it includes directions for how to install Tomcat on a server so it can be shared by multiple users across a network.

The file structure of Tomcat



The subdirectories of Tomcat

Directory	Description
bin	The binary executables and scripts
classes	Unpacked classes that are available to all web applications
common	Classes available to internal and web applications
conf	Configuration files
lib	JAR files that contain classes that are available to all web applications
logs	Log files
server	Internal classes
webapps	Web applications
work	Temporary files and directories for Tomcat

The files of Tomcat

File	Description
readme.txt	General information about Tomcat
running.txt	Instructions for installing, starting, and stopping Tomcat

Description

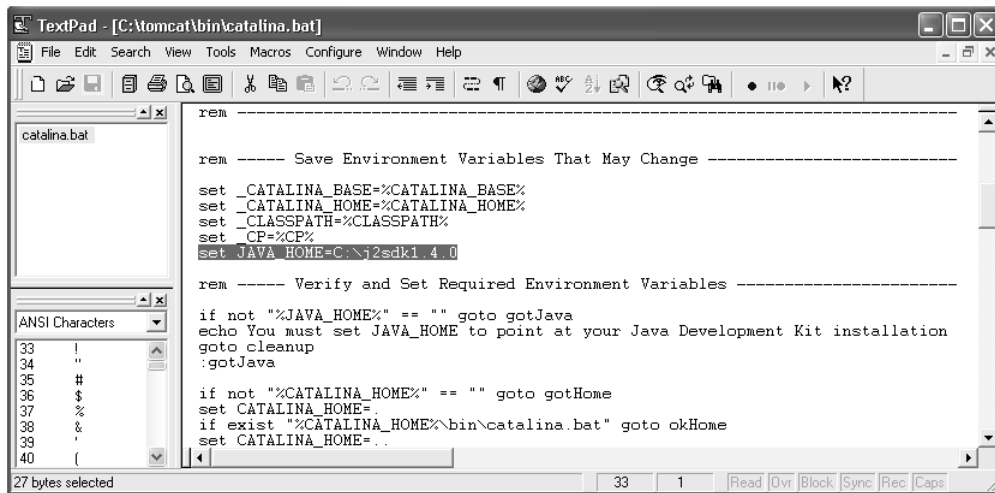
- Since Tomcat is an open-source project, developers who work on Tomcat can download the source distribution for Tomcat, modify the source code, and build Tomcat from the source code. That's why you'll find directions for how to build Tomcat on the Tomcat website.
- Developers who develop web applications only need to download the binary for Tomcat distribution and install it as described in this chapter.
- To make the classes within a JAR file available to Tomcat so you can run an application that uses them, you can put the JAR file in Tomcat's common\lib directory.

Figure 2-2 A summary of Tomcat's directories and files

How to set the JAVA_HOME environment variable

In Tomcat 4, the servlet engine is named Catalina. When Tomcat starts, it runs the `catalina.bat` file to configure this servlet engine. For the servlet engine to be able to work with Java, you must set the `JAVA_HOME` environment variable stored in the `catalina.bat` file so it points to the SDK that's installed on your system. Figure 2-3 shows how. Once you set this environment variable, Tomcat will know which version of Java to use.

The catalina batch file opened for editing



How to set the JAVA_HOME environment variable

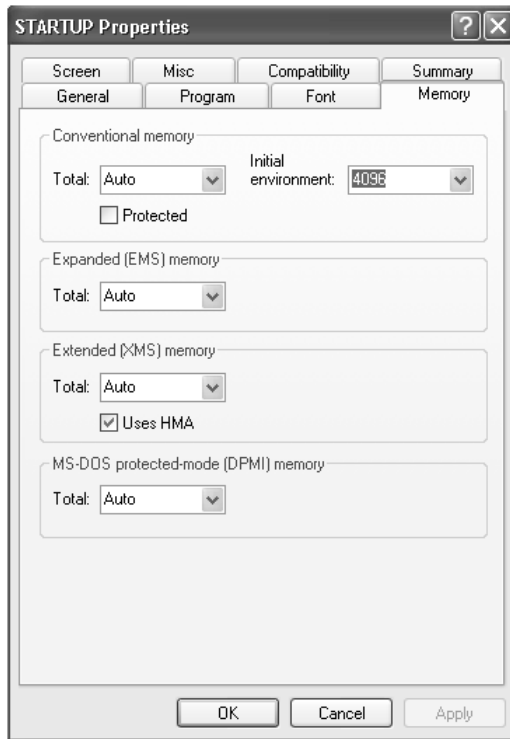
1. Use a text editor like Notepad or TextPad to open the catalina.bat file located in the c:\tomcat\bin directory. One way to do that is right-click on the name of the file in the Windows Explorer and choose Edit.
2. Scroll down to the section named “Save Environment Variables That May Change.”
3. After the last Set command, enter “set JAVA_HOME=c:\j2sdk1.4.0” where c:\j2sdk1.4.0 is the directory that contains the SDK that’s installed on your system. When you enter this command, don’t put spaces around the equal sign.
4. Save your changes to the catalina.bat file.
5. If necessary, stop and restart Tomcat.

Figure 2-3 How to set the JAVA_HOME environment variable

How to change the memory settings for Tomcat's batch files

If you're using Windows 95, 98, or ME, you may get an "out of environment space" error when you try to start Tomcat. To solve this problem, you can change the memory settings for the batch files that start and stop Tomcat as shown in figure 2-4. On the other hand, if you're using Unix or Windows NT, 2000, or XP, you shouldn't need to change the memory settings for these batch files.

The Properties dialog box for the startup batch file



How to change memory settings for the startup batch file

1. Start the Windows Explorer and navigate to the bin directory of Tomcat (c:\tomcat\bin if you changed the directory name as suggested in figure 2-1).
2. Right-click on the startup batch file and select the Properties option.
3. From the Properties dialog box, select the Memory tab.
4. Change the Initial Environment option from Auto to 4096.
5. Click on the OK button.

How to change memory settings for the shutdown batch file

- Follow the steps shown above, substituting shutdown for startup.

How to turn on servlet reloading

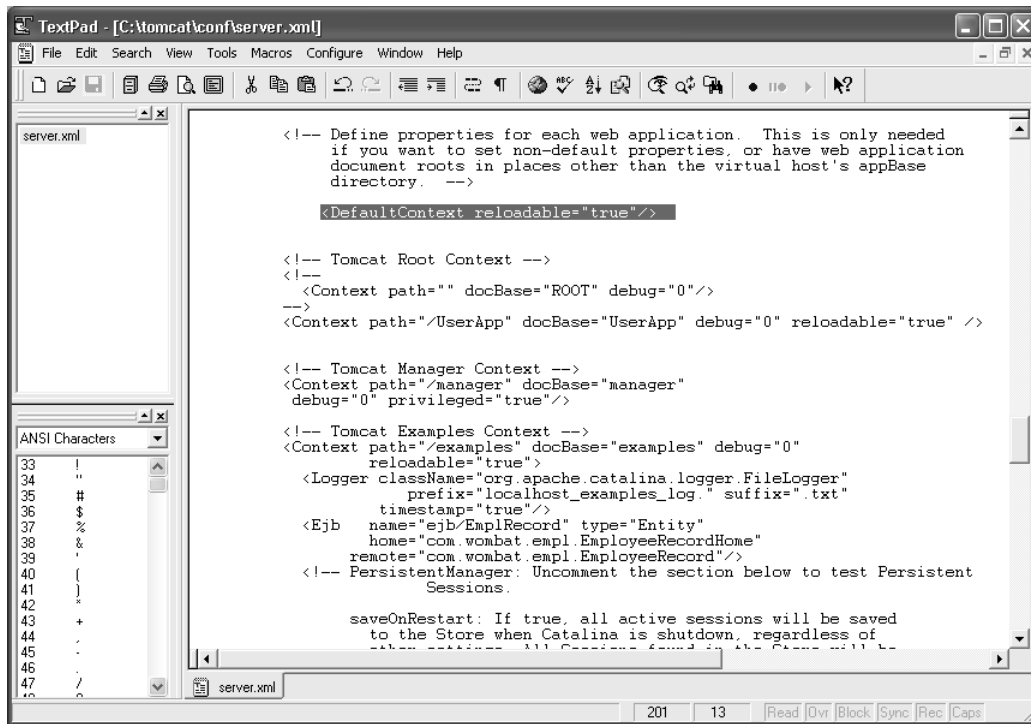
As Tomcat runs, its servlet engine loads the servlets that users request from disk into internal memory. Then, if there's another request for one of the servlets that's already in memory, Tomcat doesn't reload the servlet from disk to memory. It uses the one that's in memory.

But what if you change one of the servlets that's in memory? How does the servlet engine know that it should use the changed version of the servlet? By default, it doesn't. As a result, to test a servlet, you need to stop Tomcat and restart it each time you change a servlet. But this, of course, slows down development.

The solution is to turn on *servlet reloading*. When this option is on, Tomcat checks to make sure the requested servlet in memory is the same as the one on disk. Then, if the one on disk has been modified, it reloads the servlet before it runs it. This means that you don't have to stop and restart Tomcat each time you change a servlet.

To turn servlet reloading on, you add a line of code to the `server.xml` file that's stored in Tomcat's `conf` directory. This XML file controls how Tomcat is configured. The procedure for making this change is shown in figure 2-5.

The server.xml file



How to turn on servlet reloading

1. Use the Windows Explorer to navigate to Tomcat's conf directory and open the server.xml file in a text editor.
2. Scroll down to the comment shown above and insert the following statement below it:
<DefaultContext reloadable="true"/>
3. Save the changes to the server.xml file
4. If necessary, stop and restart Tomcat.

Description

- If *servlet reloading* isn't on, which is the default setting, you have to stop and restart Tomcat each time that you change one of the classes that's in memory.
- If you turn servlet reloading on, Tomcat checks the modification dates of the classes in memory and automatically reloads the ones that have changed so you don't have to stop and restart Tomcat each time a class changes. Although this is useful in development, it can cause performance problems in a production environment.
- The server.xml file is an XML file that controls how the Tomcat engine is configured. Tomcat reads this file every time it starts to configure itself. You can use a text editor to edit this file. Then, you can stop and restart Tomcat to put the changes into effect.

Figure 2-5 How to turn on servlet reloading

How to use Tomcat

Now that you know how to install and configure Tomcat, this topic shows you how to use it. First, you'll learn how to start and stop the Tomcat engine. Then, you'll learn how to view some HTML pages, servlets, and JSPs that come with Tomcat. This shows that Tomcat is installed correctly on your system.

How to start and stop Tomcat

Figure 2-6 shows how to start and stop Tomcat. To do that, you open up a DOS prompt window and run the startup and shutdown batch files that are stored in Tomcat's bin directory. Before you do that, though, you need to use the `cd` command to change the current directory to `tomcat\bin` so DOS will know where to find the batch files.

When you start Tomcat, several message lines are displayed in the DOS prompt window including two that show the current classpath and the Java home directory. Then, Tomcat displays a console in a second window. This console shows that Tomcat is running and is ready to receive requests for HTML pages, servlets, and JSPs.

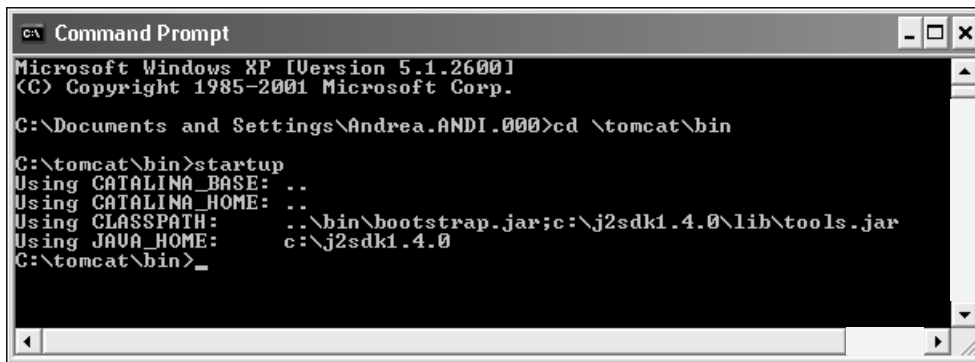
If servlet reloading is on, you only need to start Tomcat at the start of a development session, and you only need to shut it down at the end of the session. If servlet reloading isn't on, though, you need to stop and restart Java every time you want to test a newly compiled class. As a result, you'll want to leave the DOS prompt window open so you can easily start and stop Tomcat.

If you're using Unix, you can use a procedure that's similar to the one in this figure to start and stop Tomcat. After you start a Unix command prompt, you use the `cd` command to change the current working directory to Tomcat's bin directory. When using Unix, though, you use front slashes in the directory path instead of the backslashes that are required by DOS. Then, to start and stop Tomcat, you use the `startup.sh` and `shutdown.sh` commands.

When you start Tomcat for the first time, you test whether you've done the installation and configuration properly. If you haven't, one or more messages will be displayed, which should help you figure out what you did wrong. For some operating systems, though, getting everything to work right can be tedious and painstaking.

In particular, for some versions of Windows 95 and 98, we've received "out of environment space" messages, even though we've done the procedures in figure 2-4 correctly. Our solution to this problem is given in this figure, which is just another way to set the environment space variables.

DOS commands for starting Tomcat

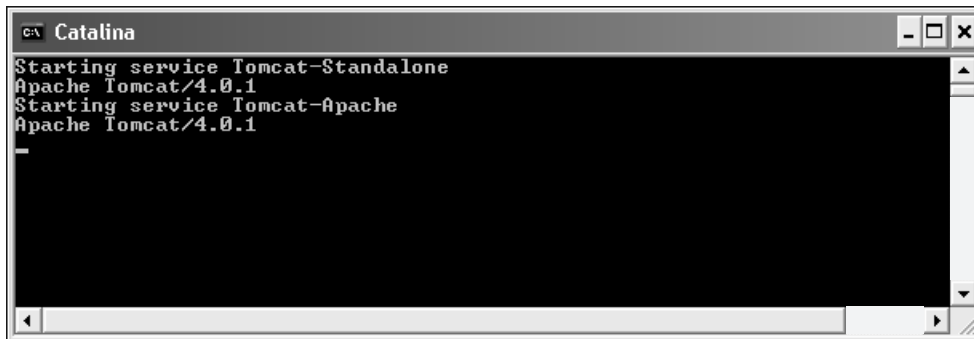


```
C:\ Command Prompt
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.

C:\Documents and Settings\Andrea.ANDI.000>cd \tomcat\bin

C:\tomcat\bin>startup
Using CATALINA_BASE:  ..
Using CATALINA_HOME:  ..
Using CLASSPATH:      ..\bin\bootstrap.jar;c:\j2sdk1.4.0\lib\tools.jar
Using JAVA_HOME:      c:\j2sdk1.4.0
C:\tomcat\bin>_
```

The console that Tomcat displays when it's running



```
C:\ Catalina
Starting service Tomcat-Standalone
Apache Tomcat/4.0.1
Starting service Tomcat-Apache
Apache Tomcat/4.0.1
_
```

How to use the DOS Prompt window to start and stop Tomcat

1. Open a DOS Prompt window and use the `cd` command to change the current directory to Tomcat's `bin` directory.
2. To start Tomcat, type “`startup`” and press the Enter key. To stop Tomcat, type “`shutdown`” and press the Enter key.

What to do if Tomcat doesn't start because it's “out of environment space”

- First, make sure that you did the procedures in figure 2-4 correctly. If you did and you're using Windows 95 or 98, you can then set these variables this way: (1) click on the Title bar of the DOS Prompt window; (2) select Properties from the shortcut menu; (3) click on the Memory tab; and (4) change the Initial environment setting to 4096. After you close and reopen the DOS Prompt window, this should fix the problem.

How to view a web page

On most systems, you can view an HTML document without using a web server. To do that, you can use the Windows Explorer to navigate to the HTML file for the page and double-click on it. This opens up your web browser, which displays the HTML document. Another way to view an HTML document is to open up your web browser and enter the path and filename for the HTML file.

To view a web page that's generated by a JSP or servlet, though, you need to use HTTP to request the web page through Tomcat as shown in figure 2-7. After you start Tomcat and your browser, you enter a *Uniform Resource Locator*, or *URL*, in the browser. Then, Tomcat finds the servlet for the page that you've requested, runs the servlet to generate the HTML for the page, and sends the HTML back to your browser.

The Address box of the Internet Explorer browser that's shown in this figure contains a typical URL. This URL requests an HTML page that's part of a web application. Here, the URL begins with the *protocol*, which is Hypertext Transfer Protocol, or HTTP. The protocol is followed by a colon and two slashes.

After the protocol, the URL must specify the *host*, which is the server that's hosting the web application. To specify the local system, you can use the localhost keyword. To specify another computer, though, you usually use an address that includes the domain name like *www.murach.com*. The alternative is to use a specific IP address like 64.71.179.86, but you rarely have to do that.

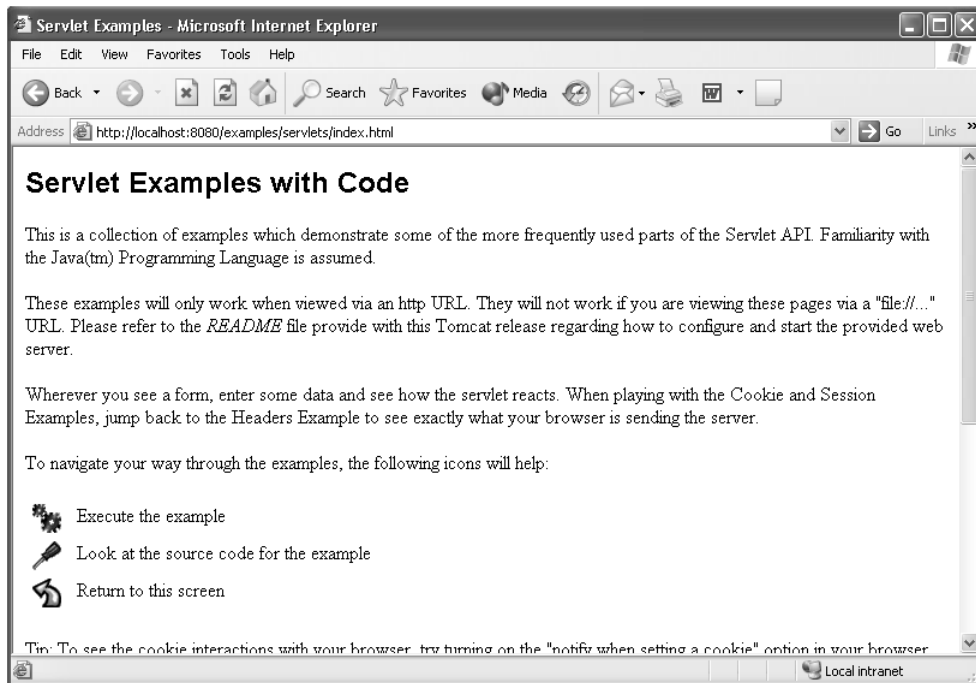
After the host, the URL must specify the *port*. By default, the port for Tomcat is 8080. If another application uses the same port, though, you may need to change that port number as shown in figure 2-10.

After the port, you specify the *path* and the *filename* of the resource. In this example, the web browser has requested the *index.html* file that's stored in Tomcat's *examples/servlets* directory. Please note here that front slashes are used to separate the components of a path in a URL, although backslashes are used to separate them in a DOS path.

If you display the page shown in this figure and read it, you'll see that it lets you run several of the servlet examples that come with Tomcat. To make sure that the Tomcat servlet engine is installed properly on your system, you can run some of them. If, for example, you scroll down to the Hello World application and click on the Execute link, the application should use a servlet to display the words "Hello World" in your browser. This shows that your servlet engine is working properly.

Similarly, if you display the index page for the *examples\jsp* directory, you'll see a page that lets you run several of the JSP examples that come with Tomcat. To make sure that the JSP engine is configured properly, you can run some of these examples.

A web browser that displays a web page



The components of an HTTP URL

http://localhost:8080/examples/servlets/index.html

protocol	host	port	path	filename

How to view a web page via an HTTP URL

1. Start Tomcat.
2. Start your web browser.
3. In the Address text box (for the Internet Explorer) or Location text box (for Netscape), type a URL and press Enter.

Description

- You can use the syntax shown above to specify a *Uniform Resource Locator*, or *URL*.
- When Tomcat is running on your local machine, you can use the "localhost" keyword to specify the host machine.
- The default port for Tomcat is 8080. If another application is already using this port, you can change the default port as shown in figure 2-10.

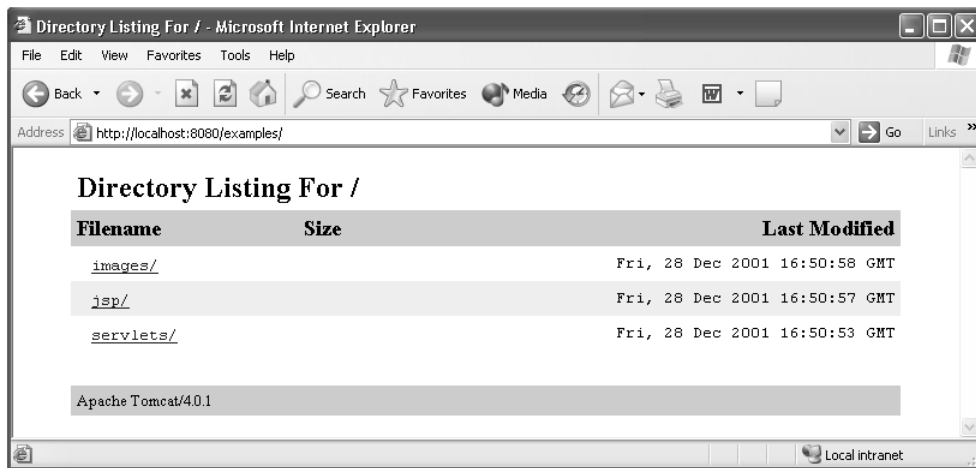
Figure 2-7 How to view a web page via an HTTP URL

How to view a directory listing

Figure 2-8 shows how to view a directory listing. To start, you enter the URL for the directory into your browser. Then, if the directory contains a file named `index.htm`, `index.html`, or `index.jsp`, Tomcat will display the index file for the directory. Otherwise, Tomcat will display a directory listing like the one shown in this figure. You can use the links in this listing to navigate to other directory listings or web pages.

Since the `examples\servlets` directory in the last figure contains an index file named `index.html`, you can't display a directory listing for that directory. However, since the `examples` directory shown in this figure doesn't contain an index file, you can display a directory listing for this directory. Then, you can click on the `servlets` link to navigate to the `examples\servlets` directory, which displays the index page for the servlet examples, or you can click on the `jsp` link to navigate to the `examples\jsp` directory, which displays the index page for the JSP examples.

A web browser that displays a directory listing



Description

- If a URL specifies a directory but not a filename, most web servers will automatically display any file within that directory named index.htm, index.html, or index.jsp. However, if that directory doesn't contain an index file, most web servers will display a directory listing like the one shown above.
- To navigate to the examples\servlets or examples\jsp directories from this web page, you can click on one of the links.

Figure 2-8 How to view a directory listing via an HTTP URL

A quick guide to troubleshooting

This topic presents a quick guide to troubleshooting some common Tomcat problems. For more help, you can consult the Apache web site.

Two common Tomcat problems and how to solve them

The first error page shown in figure 2-9 indicates that there's a problem connecting the browser to Tomcat. If you're using a different version of Internet Explorer, you'll see a similar error page. And if you're using the Netscape browser, you'll get a message that says that the browser can't locate the server.

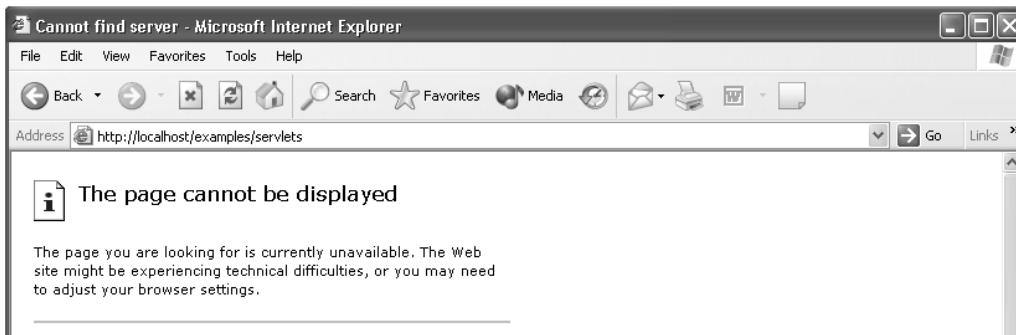
No matter how the message looks, there are two probable causes for this problem. First, the server might not be running, which you can fix by starting Tomcat. Second, you may have entered the URL incorrectly, which you can fix by making sure that the URL points to a valid web server. In this example, the user didn't enter the port correctly, which is a common problem.

If neither of these approaches solves the problem, it may be because another web server or process is trying to use the same port as Tomcat. If so, you'll need to change the port used by Tomcat as shown in the next figure.

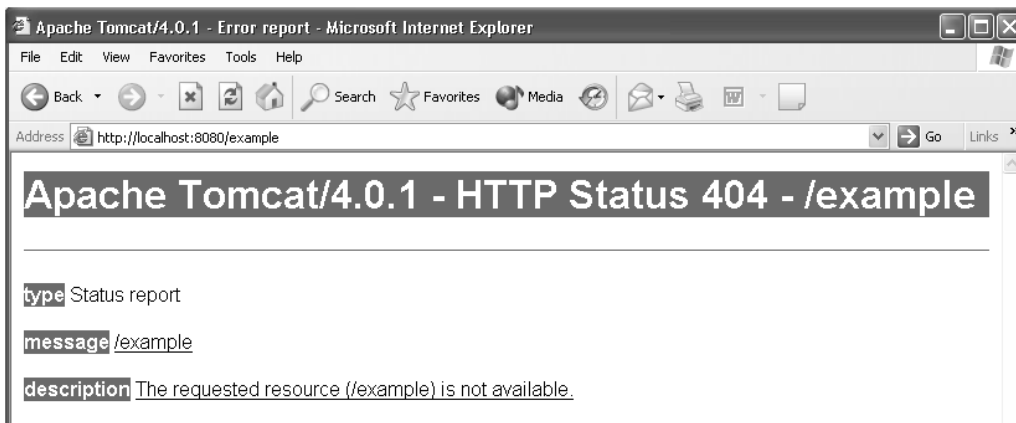
The second error page in this figure shows that the browser has successfully connected to the server, but the server can't find the page requested by the browser. This is the default error page that Tomcat uses for this problem. If you're running Tomcat on a local Windows machine, you can use the Windows Explorer to make sure the directory in the URL really exists.

Since it's a common practice to substitute a custom error page for the default error page, the error page that you get when you encounter a 404 error might look different than the one in this figure. Later in this book, you'll learn how to create your own custom error page and how to configure Tomcat to display that page when it encounters various types of errors.

The Internet Explorer's error page



Tomcat's default 404 error page



Description

- If the browser displays a message like the first error page, the HTTP request isn't connecting with a web server. To solve this problem, make sure that the Tomcat engine is running, and make sure that you've entered a valid URL.
- If the browser displays a Tomcat error page like the second error page, Tomcat is receiving the HTTP request, but it can't find the requested resource. To solve this problem, make sure that you've entered the path and filename of the URL correctly.
- The second error page is the default 404 error page that's returned by Tomcat when it can't find the requested resource. However, it's possible to configure Tomcat so it displays a custom error page instead of the default error page.

How to change the port that's used by Tomcat

By default, Tomcat uses port 8080 as shown throughout this chapter. However, if you have a port conflict with another application, you may need to change that port as shown in figure 2-10. Here again, you modify the `server.xml` file that's stored in Tomcat's `conf` directory. In particular, you replace all instances of 8080 with another four-digit number that's greater than 1024.

After you save the `server.xml` file and stop and restart the server, the new port should take effect. If, for example, you change from port 8080 to 1979 as shown in this figure, you can access the directory listing for Tomcat's examples by entering this URL:

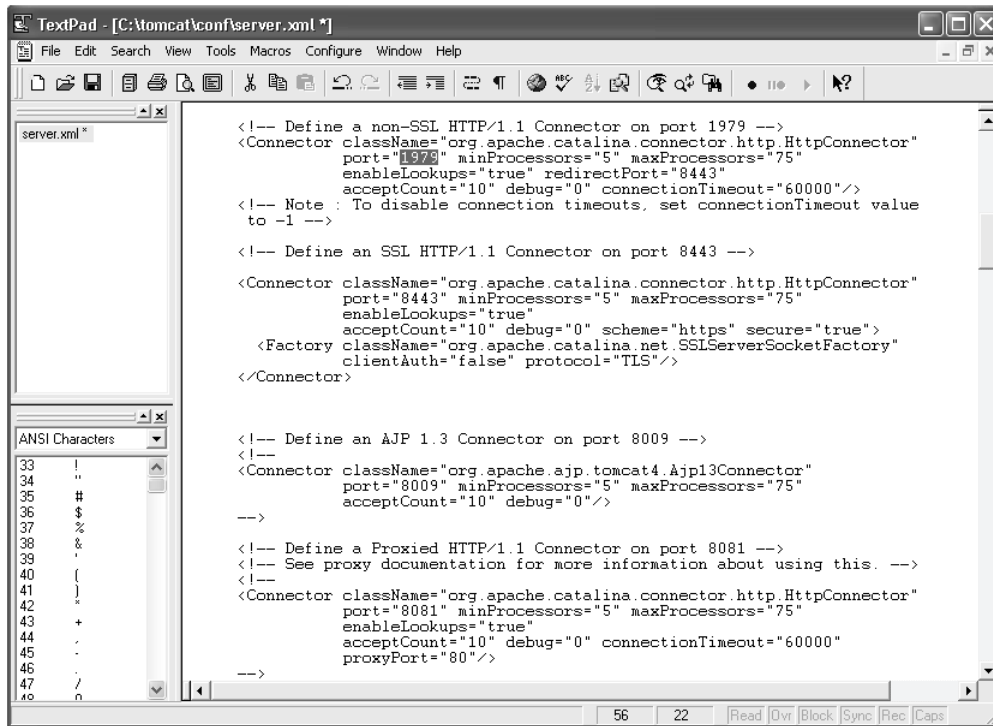
`http://localhost:1979/examples`

Because port 80 is the default port for most browsers, another alternative is to change the port from 8080 to 80. Then, you don't have to enter the port number in your URLs as in this example:

`http://localhost/examples`

Since this can save some typing, you may want to try this. However, if this causes a port conflict, you will have to change to another port.

The server.xml file



How to change the port that's used by Tomcat

1. Use the Windows Explorer to navigate to Tomcat's conf directory and open the server.xml file in a text editor.
2. Replace all instances of the current port, which is 8080 by default, to a four-digit number that's greater than 1024 or to 80. To do this, you may want to use the Find and Replace feature of your text editor.
3. Save the changes to the server.xml file.
4. Stop and restart Tomcat.

Description

- If you have a port conflict with another application, you can change the default port from 8080 to a four-digit number that's greater than 1024.
- If you don't enter a port when you specify a URL, your browser will use port 80. As a result, if you change Tomcat's default port from 8080 to 80, you don't need to enter a port when entering a URL in the browser. Then, assuming that there isn't a port conflict, you can view a web page by entering a URL like this:

http://localhost/examples/servlets

Figure 2-10 How to change the port that's used by Tomcat

How to deploy and run a web application

Once you install Tomcat, you can deploy a web application and run it. To start, you can deploy the web applications that are included on the CD that comes with this book. This should help you understand how J2EE web applications are structured.

How to deploy a web application

To deploy a web application, you copy the files for the web application to the appropriate directories on the web server. To deploy a web application that's running on the Internet, you need to use an FTP program to upload your files to the appropriate directories on the web server. But to deploy a web application that's running on an intranet or on your own computer, you can use a tool like the Windows Explorer to copy the files to the appropriate directories.

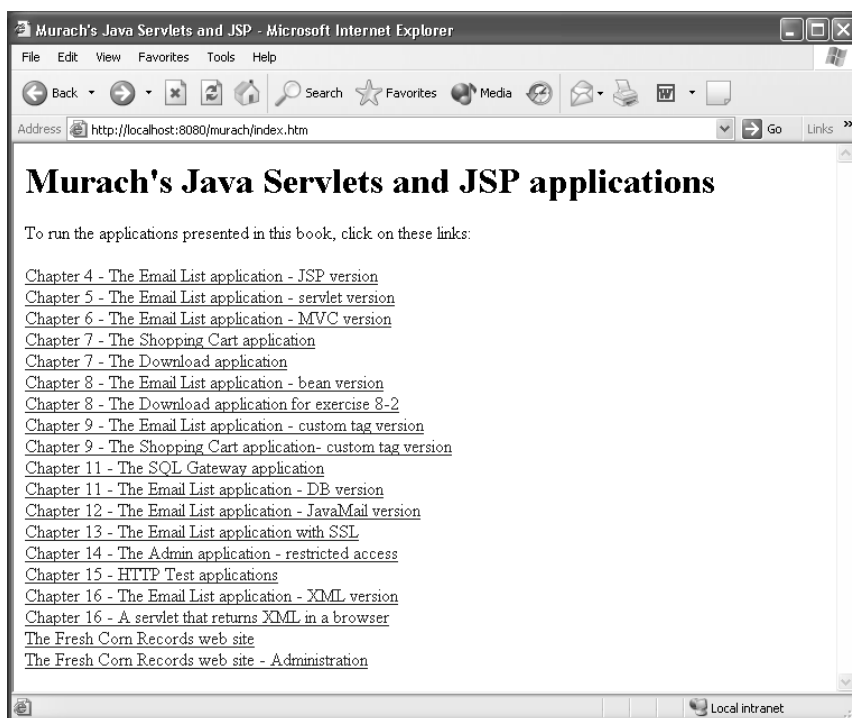
Figure 2-11 shows how to install the web applications that are included on the CD that comes with this book. These web applications include the examples that are presented in each chapter, plus an online music store application that's presented in the last four chapters of this book. To start, you run the Install.exe program that's on the CD. That should copy all of the files for the web applications to the c:\murach\webapps directory on your hard drive. Then, to install the web applications in the appropriate directories for Tomcat, you copy (not move) the murach and musicStore directories from the c:\murach\webapps directory to Tomcat's webapps directory.

How to view and run a web application

Once you've copied the files for a web application to the appropriate directory, you can view and run the pages for the web application by entering a URL that points to a web page or by clicking on a link that points to a web page. In this figure, the URL points to the index page for the murach directory that's stored in Tomcat's webapps directory. Then, each link on this web page points to a web application that's used in one of the chapters of this book.

Since you will probably want to visit this page often as you work through this book, you may want to use your web browser to bookmark this page. Then, you'll be able to access this page without having to type the complete URL each time.

The index page for the webapps\murach directory



How to install the web applications that are on the book's CD

1. From the root directory of the CD, double-click on the file named Install.exe and respond to the dialog boxes that follow. This will install the source code and test files in the c:\murach\webapps directory.
2. Copy the c:\murach\webapps\murach directory and all its subdirectories to c:\tomcat\webapps.
3. Copy the c:\murach\webapps\musicStore directory and all its subdirectories to c:\tomcat\webapps.

How to view and run the web applications that are on the book's CD

- To view the index page for all applications stored under the murach directory, start Tomcat and access <http://localhost:8080/murach> from your browser. Then, click on the link for the application that you want to run.
- To view the index page for the musicStore directory, start Tomcat and access <http://localhost:8080/musicStore> from your browser. This will display the index page for the Music Store application that's presented in this book.

Note

- Appendix A presents the procedures that you need for installing all of the components that the CD contains. Once you install these components, including the murach and music databases, all of the web applications for the book should work.

Figure 2-11 How to install and run the book applications

A summary of the directories and files for a web application

Figure 2-12 shows a simplified version of the directory structure for the Music Store application that's presented in this book. Some of these directories are standard directories that are used by all J2EE web applications. For example, all web applications that use servlets must have the WEB-INF and WEB-INF\classes directories. In addition, though, some of the directories are defined by the programmer to help structure the web application. In particular, the admin, cart, and download directories are used to contain the Administration, Shopping Cart, and Download applications that make up the Music Store application.

To start, each web application must have a root directory that can be referred to as the *document root directory*, or just *document root*. In this figure, the document root directory is named musicStore, and it is subordinate to Tomcat's webapps directory. Then, all of the other directories and files for the application must be subordinate to this document root.

The WEB-INF directory that's subordinate to the document root directory usually contains just one file named web.xml. This file is used to configure the servlets and other components of the web application. You'll learn more about it in the next figure.

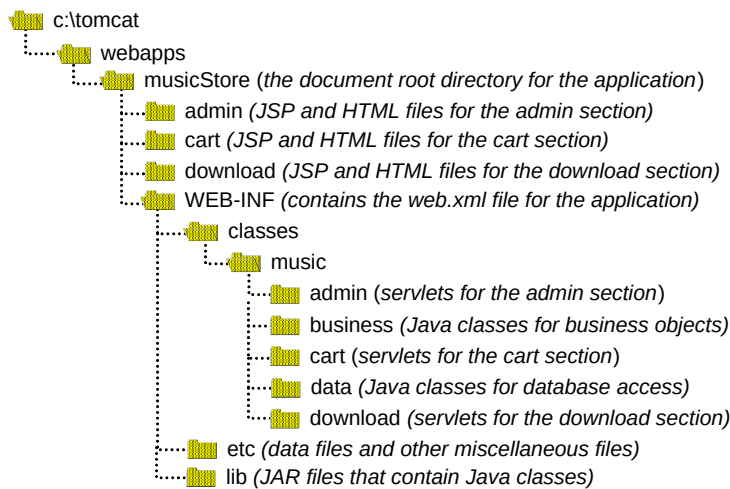
The WEB-INF directory also has a few standard directories that are subordinate to it. In particular, the WEB-INF\classes directory contains the servlets and other Java classes for the application that aren't compressed into JAR files. These are typically the classes that you write. In contrast, the WEB-INF\lib directory contains the JAR files for the application, and the JAR files that apply to all the applications that Tomcat is running must be stored in the webapps\common\lib directory. In case you need to refresh your memory, a *JAR file* contains a group of classes in a compressed format.

To organize the classes that you create for the application, you can store them in *packages*. In that case, you need to create one subdirectory for each package. For example, the application in this figure uses five packages: admin, business, cart, data, and download. Three of these packages contain servlets that work with the HTML and JSP files for the application while the other two contain the Java classes that provide the business objects and data access objects for the web application.

Besides the standard WEB-INF subdirectories, the Music Store application uses the WEB-INF\etc directory to store all of the data files that it uses. Since the WEB-INF directory and its subdirectories aren't web accessible, this prevents users from accessing these files with a web browser.

As you progress through this book, you'll learn how to use some other standard directories to deploy web applications. For example, if you use custom tags, you'll need to include the WEB-INF\tlds directory. For now, though, this figure should give you a general idea of what it takes to deploy a web application.

The directory structure for a web application named musicStore



A summary of the directories and files for a web application

Directory	Description
document root	The <i>document root directory</i> is the root directory for a web application. This directory typically contains the index file for the application. In addition, this directory or its subdirectories contain the HTML and JSP files for the application.
\WEB-INF	This directory contains a file named web.xml. It can be used to configure the servlets and other components that make up the application.
\WEB-INF\classes	This directory and its subdirectories contain the servlets and other Java classes for your application that aren't compressed into JAR files. If you're using Java packages, each package must be stored in a subdirectory that has the same name as the package.
\WEB-INF\lib	This directory contains any JAR files that contain Java classes that are needed by this web application, but not by other web applications.

Description

- A Java web application is a hierarchy of directories and files in a standard layout defined by the J2EE specification. This means that each Tomcat web application must use the four directories summarized above.
- To organize your Java class files, you can store them in *packages*. A package of classes must be stored within a subdirectory of the WEB-INF\classes directory.
- If you need to make classes within a JAR file available to more than one web application, you can put the JAR file in Tomcat's webapps\common\lib directory. If you need to make classes within a JAR file available to a single web application, you can put them in the WEB-INF\lib directory for that application.
- You can use the WEB-INF\etc directory to store any files that you don't want users to be able to access. For example, you can store any files that store data for the application in this directory.

Figure 2-12 A summary of the directories and files for a web application

An introduction to the web.xml file for a web application

Figure 2-13 introduces you to the web.xml file that is known as the *deployment descriptor*. This file is used to configure a web application, and this figure should give you a general idea of what the web.xml file can do for an application. At the minimum, this file must contain the shaded code. This code defines the type of XML that's being used and the type of J2EE standards that the web.xml file follows.

Typically, though, a web.xml file contains some additional code that's used to configure the web application. In this figure, for example, the XML tags within the Servlet tags specify an alias and some initialization parameters for a servlet. In particular, they specify an alias for the EmailServlet that's stored in the email package so it can be accessed by the name EmailResponse. And they specify an initialization parameter that says to use the UserEmail.txt file for a parameter named filename.

As you progress through this book, you'll learn some specifics for working with a web.xml file. In chapter 6, for example, you'll learn how to use a web.xml file to work with servlet aliases, servlet mapping, initialization parameters, and custom error pages. And in chapter 14, you'll learn how to use the web.xml file to restrict access to the pages within a web application.

A web.xml file

```
<?xml version="1.0" encoding="ISO-8859-1"?>

<!DOCTYPE web-app
  PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.2//EN"
  "http://java.sun.com/j2ee/dtds/web-app_2_2.dtd">

<web-app>

  <servlet>
    <servlet-name>EmailResponse</servlet-name>
    <servlet-class>email.EmailServlet</servlet-class>
    <init-param>
      <param-name>filename</param-name>
      <param-value>UserEmail.txt</param-value>
    </init-param>
  </servlet>

</web-app>
```

What the web.xml file can do

- Provide an alias for a servlet class so you can call a servlet using a different name.
- Enable servlet mapping so you can call a servlet using any URL or URL pattern.
- Define initialization parameters for a servlet or the entire application.
- Define error pages for an entire application.
- Provide security constraints to restrict access to certain web pages and servlets.

Description

- Every web application requires a web.xml file in the WEB-INF directory. This file is known as the *deployment descriptor* for the web application. At the minimum, this file must contain the information that's highlighted above.
- Throughout this book, you'll learn how to manipulate the web.xml file to implement the tasks described above.

A summary of the directories and files for the book applications

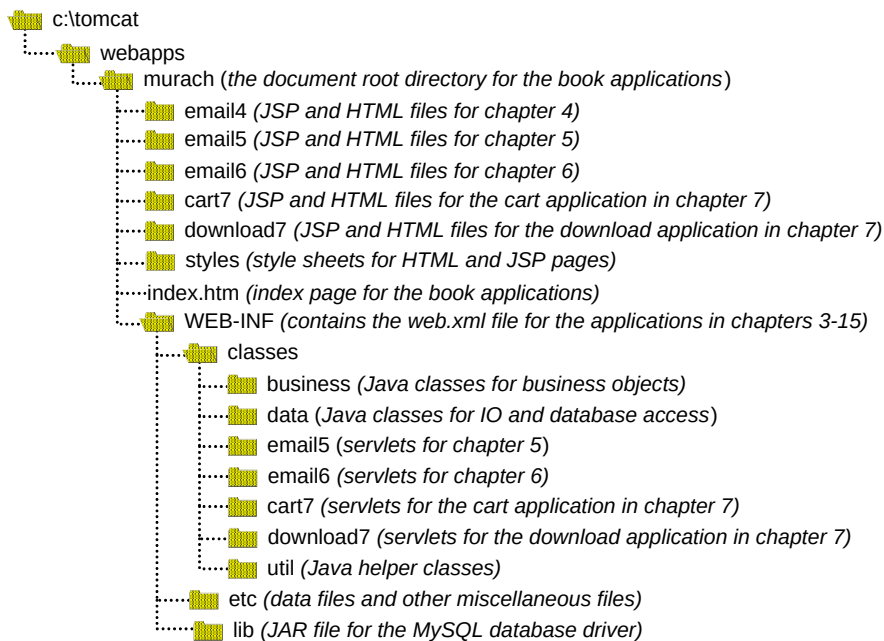
In figure 2-12, you saw the directory structure for a web application. Now, in figure 2-14, you can see the directory structure for the many small applications that are used to illustrate the skills presented in chapters 3 through 16. Once you understand this structure, it will be easy for you to find the parts of these applications that you want to review.

In this case, the document root directory is the `murach` directory. Then, for each chapter, the JSP and HTML files are stored in a subdirectory like `murach\email6`, and the servlet classes are stored in a subdirectory of the classes directory like `WEB-INF\classes\email6`.

Beyond that, the classes that can be used by any of the chapter applications are stored in packages in subdirectories like `classes\business`, `classes\data`, and `classes\util`. The data files that can be used by all the applications are stored in the `WEB-INF\etc` directory. And the JAR file for the MySQL database driver is stored in the `WEB-INF\lib`.

Please note that the directory structure in this figure doesn't include all of the directories. But with this as background, you should be able to find any of the files that you install from this book's CD.

The directory structure for the book applications



Description

- In chapters 3 through 16, you'll learn how to develop the web applications that you install from this book's CD. Unlike the Music Store application that's summarized in figure 2-13, these applications are short applications that are designed to illustrate the skills presented in each chapter.
- The document root directory for all of these applications is webapps\murach. And the index file that provides links to these applications is in the murach directory.
- The HTML documents, JSPs, and servlets for each of these applications are stored in specific directories. For instance, the HTML and JSPs for the chapter 6 application are stored in the murach\email6 directory, and the servlet classes for this chapter are stored in the murach\WEB-INF\classes\email6 directory.
- Files that aren't specific to a chapter are stored in subdirectories of the murach directory.
- Java classes that are available to all applications are stored in separate packages in the WEB-INF/classes directory.
- Although the directory structure above doesn't include all of the directories for the book applications, you can view the complete structure after you install the book applications as shown in figure 2-11.

Figure 2-14 A summary of the directories and files for the book applications

Perspective

The goal of this chapter has been to show you how to install, configure, and use Tomcat on your own PC. Now, if you can use Tomcat to run the web applications that are included on the book's CD, that goal has been fulfilled. In that case, you're ready to move on to the next chapter where you'll learn how to develop your own HTML documents.

Summary

- After you install Tomcat on your own PC, you need to copy the `servlet.jar` file from Tomcat's `common\lib` directory to the SDK's `jre\lib\ext` directory. You also need to set the `JAVA_HOME` environmental variable so it points to the SDK that's installed on your PC.
- If you want Tomcat to recognize changes that are made to the servlet classes that are used, you need to turn on *servlet reloading* by modifying the `servlet.xml` file. Then, you don't have to stop and restart Tomcat every time you change a class.
- Before you can run servlets and JSPs on your own system, you need to open a DOS prompt window and use the startup batch file to start Tomcat. When you're done with Tomcat, you use the shutdown batch file to turn Tomcat off.
- When Tomcat is running, you can use your web browser to enter a *Uniform Resource Locator (URL)* that accesses an HTML document, a JSP, or a servlet. If you enter a URL that points to a directory that doesn't contain an `index.htm`, `index.html` file, or `index.jsp`, Tomcat displays a directory listing.
- Two common problems that occur when you enter a URL into your browser are (1) that Tomcat isn't running so the browser can't connect with the web server, and (2) that the URL is incorrect so Tomcat can't find the page you want. In either case, an error message is displayed.
- If Tomcat's default *port* is used by another application on your system, you may need to change the default port by modifying the `servlet.xml` file.
- A Java web application consists of a hierarchy of directories and files in a standard layout that's defined by the J2EE specification. This layout includes a *document root directory*, a `WEB-INF` directory, and `classes` and `lib` directories that are subordinate to the `WEB-INF` directory.
- The *deployment descriptor* for a web application is a `web.xml` file in the `WEB-INF` directory that can be used to configure the servlets and other components that make up the application.

Terms

Tomcat home directory
servlet reloading
Uniform Resource Locator (URL)
protocol
host
port
path
filename
document root directory
document root
JAR file
package
deployment descriptor

Objectives

- Install and configure Tomcat so it runs on your own PC.
- Start Tomcat from a DOS prompt window, and shut it down from this window.
- Turn on servlet reloading so you don't have to stop and restart Tomcat every time you want to change and test a servlet.
- Use your browser to enter a URL that accesses an HTML document, JSP, or servlet.
- If a port conflict occurs, change the port that's used by Tomcat.
- Describe two common errors that occur after you enter a URL into a browser.
- In general terms, describe the directory structure of a web application that's defined by the J2EE specification.
- In general terms, describe the deployment descriptor of a web application.

Before you do the exercises for this chapter

If you haven't already done so, you need to install all of the software that's on the book's CD. You should also install the web applications that are on the CD. In appendix A, you'll find the procedures that you need for installing all of the software and applications, except for Tomcat.

Exercise 2-1 Install and test Tomcat

If you've installed and tested Tomcat as you went through this chapter, you don't need to do this exercise.

1. Install and configure Tomcat as described in this chapter.
2. To make sure you've installed Tomcat correctly, start Tomcat and run some of the Tomcat examples as shown in figure 2-7.
3. Shutdown Tomcat.

Exercise 2-2 Run the book applications

Before you do this exercise, you need to install the book applications from this book's CD as described in appendix A.

1. Display the index page for the murach book applications, and run any of the applications for chapters 4 through 16. If you like, bookmark the index page for these applications.
2. Start the Music Store application by entering the URL for its index page, and experiment with this application to make sure that the MySQL database has been installed correctly.
3. Use the Windows Explorer to review the directories and files for the murach book applications and the Music Store application.